

**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE
SÃO PAULO**

Campus Campos do Jordão

Daniel Eduardo Machado

**SISTEMA DE AUTOATENDIMENTO BANCÁRIO (ATM) EM
C++11 COM INTERFACE GRÁFICA QT**

CAMPOS DO JORDÃO

2026

Dados do Relatório:

- **Instituição de Ensino:** IFSP - Campus Campos do Jordão
- **Curso:** Análise e Desenvolvimento de Sistemas
- **Disciplina:** Programação Orientada a Objetos
- **Professor:** Paulo Giovani de Faria Zeferino
- **Aluno:** Daniel Eduardo Machado

RESUMO

O presente trabalho descreve o desenvolvimento de um simulador de terminal de autoatendimento bancário (ATM) utilizando a linguagem C++ sob o padrão C++11 e o framework Qt para a interface gráfica. O objetivo principal do projeto consiste em criar uma aplicação desktop robusta que emule operações bancárias reais, incluindo consulta de saldo, extrato detalhado, depósitos, saques e pagamento de contas com validação de código de barras. A metodologia adotada baseia-se no paradigma de Programação Orientada a Objetos (POO), empregando o padrão de Máquina de Estados Finitos para o gerenciamento do fluxo de telas e sessões de usuário, além de estruturar a separação clara entre as camadas de dados, controle e visão (interface). Os resultados obtidos demonstram uma aplicação funcional que valida credenciais de acesso de forma segura, calcula transações de maneira atômica e registra o histórico de operações com persistência de dados em disco. Conclui-se que a integração do C++ com as bibliotecas e ferramentas de design do Qt, especificamente o uso de Qt Style Sheets (QSS) para simular o Fluent Design do sistema operacional Windows 11, fornece uma solução eficiente, moderna e estável para a simulação de sistemas financeiros críticos.

Palavras-Chave: Caixa eletrônico; Programação orientada a objetos; Framework Qt; C++11; Sistemas financeiros.

ABSTRACT

The present work describes the development of an automated teller machine (ATM) simulator using the C++ language under the C++11 standard and the Qt framework for the graphical user interface. The main objective of the project is to create a robust desktop application that emulates real banking operations, including balance inquiry, detailed statements, deposits, withdrawals, and bill payments with barcode validation. The methodology adopted is based on the Object-Oriented Programming (OOP) paradigm, employing the Finite State Machine pattern to manage the screen flow and user sessions, in addition to structuring a clear separation between the data, control, and view (interface) layers. The obtained results demonstrate a functional application that securely validates access credentials, atomically calculates transactions, and records the operation history with data persistence on disk. It is concluded that the integration of C++ with Qt libraries and design tools, specifically the use of Qt Style Sheets (QSS) to simulate the Fluent Design of the Windows 11 operating system, provides an efficient, modern, and stable solution for simulating critical financial systems.

Keywords: Automated teller machine; Object-oriented programming; Qt framework; C++11; Financial systems.

LISTA DE SIGLAS

ATM - Automated Teller Machine (Caixa Eletrônico)

MVC - Model-View-Controller

POO - Programação Orientada a Objetos

QSS - Qt Style Sheets

UI - User Interface (Interface do Usuário)

1 INTRODUÇÃO

Neste capítulo serão introduzidos todos os assuntos abordados por este documento. Pretende-se apresentar a motivação, os objetivos e a organização do texto referente ao desenvolvimento do simulador de terminal de autoatendimento bancário (ATM).

1.1 Objetivos

Este trabalho tem por objetivo desenvolver um simulador desktop completo de Caixa Eletrônico (ATM) operando em ambiente Windows 11.

Para a consecução deste objetivo principal foram estabelecidos os seguintes objetivos específicos:

Construir um banco de dados em memória para gerenciar contas bancárias e persistir os dados em disco.

Implementar a lógica de negócios para operações de saque, depósito, consulta de saldo e pagamentos de boletos com validação de código de barras.

Desenvolver uma interface gráfica reativa e amigável utilizando o framework Qt, simulando o visual moderno (Fluent Design) do Windows 11.

1.2 Justificativa

A relevância deste trabalho se dá pela necessidade de aplicar, em um cenário prático e complexo, os conceitos avançados de engenharia de software e arquitetura de sistemas. A construção de um ATM exige o tratamento rigoroso de dados financeiros, controle estrito de estados do sistema (evitando fraudes ou falhas durante transações) e uma interface de fácil usabilidade, refletindo os desafios reais encontrados na indústria de desenvolvimento de software corporativo.

1.3 Aspectos Metodológicos

O presente estudo fez uso de pesquisa bibliográfica para o embasamento teórico sobre arquitetura de software e C++11. Para a parte prática, adotou-se o desenvolvimento iterativo, iniciando pela modelagem do sistema através de diagramas UML (Classes e Estados), seguido da codificação da lógica de backend, construção da interface no Qt Designer e, por fim, a integração e testes manuais de validação das transações bancárias.

1.4 Aporte Teórico

Na seção 2 são apresentadas as principais bases teóricas que fundamentaram este trabalho, focando em POO e no ecossistema Qt. A seção 3 apresenta a metodologia de desenvolvimento, a estrutura de pastas e a arquitetura do sistema proposto. Na seção 4 é apresentada a avaliação do sistema desenvolvido. Por fim, a seção 5 apresenta a conclusão deste trabalho.

2 FUNDAMENTAÇÃO TEÓRICA

Nesta seção será apresentada uma revisão da literatura pertinente às tecnologias e conceitos utilizados no desenvolvimento do trabalho.

2.1 Programação Orientada a Objetos em C++

A POO é um paradigma de programação baseado no conceito de "objetos", que podem conter dados na forma de campos (atributos) e códigos na forma de procedimentos (métodos). Segundo Stroustrup (2014), a linguagem C++ suporta a abstração de dados, programação orientada a objetos e programação genérica, permitindo ao desenvolvedor criar sistemas robustos com forte encapsulamento. No contexto deste projeto, conceitos como polimorfismo foram essenciais para tratar diferentes tipos de transações financeiras (saques, depósitos) sob uma classe base comum.

2.2 Framework Qt e Design de Interfaces

O Qt é um framework de desenvolvimento multiplataforma amplamente utilizado para a criação de interfaces gráficas de usuário (GUI). Conforme a

documentação da The Qt Company (2026), o mecanismo de Signals e Slots é a base para a comunicação entre objetos no Qt, substituindo as tradicionais funções de callback e permitindo um acoplamento fraco entre a interface gráfica e a lógica de negócios subjacente.

2.3 Trabalhos Relacionados

Sistemas de simulação de caixas eletrônicos são comumente utilizados como projetos acadêmicos para demonstrar a aplicação de padrões de projeto de software (Design Patterns). Diferente de implementações puramente baseadas em terminal (console), este projeto destaca-se por emular uma máquina de estados finita atrelada diretamente a uma interface gráfica rica, gerenciando a transição de telas e a persistência de arquivos binários simultaneamente.

3 PROJETO PROPOSTO (METODOLOGIA)

Nesta seção serão apresentadas detalhadamente a arquitetura do sistema proposto, as decisões de projeto e como a implementação foi estruturada para atender aos requisitos de um ATM bancário.

3.1 Considerações Iniciais e Arquitetura

A arquitetura do projeto foi desenhada visando a máxima separação de responsabilidades. O sistema é orquestrado pela classe principal `ATMController`, que atua como ponte entre a interface do usuário (`MainWindow` no Qt) e a base de dados emulada (`BankDatabase`). As operações foram modeladas de forma que o banco de dados seja a única entidade capaz de alterar o saldo das contas, garantindo a integridade transacional.

3.2 Requisitos

O sistema atende aos seguintes requisitos funcionais:

Validação de credenciais (Número da conta e PIN).

Bloqueio de acesso após falhas de autenticação.

Execução de saques com validação de saldo disponível.

Depósitos simulados (dinheiro e cheque).

Pagamento de contas condicionado à validação de formato do código de barras (Expressões Regulares - Regex).

Geração de extrato detalhado com data, operação, valor e saldo resultante.

3.3 Implementação e Separação de Camadas

A organização do projeto foi estruturada nos seguintes diretórios para manter a escalabilidade:

/include e /src: Contêm os cabeçalhos (.h) e códigos-fonte (.cpp) das classes Account, BankDatabase e Transaction. O polimorfismo é aplicado na classe Transaction, da qual derivam operações específicas como Withdrawal e BillPayment.

/ui: Armazena o arquivo XML mainwindow.ui, onde a interface foi desenhada utilizando um componente QStackedWidget para alternar fluidamente entre as telas de ociosidade, autenticação e menu principal.

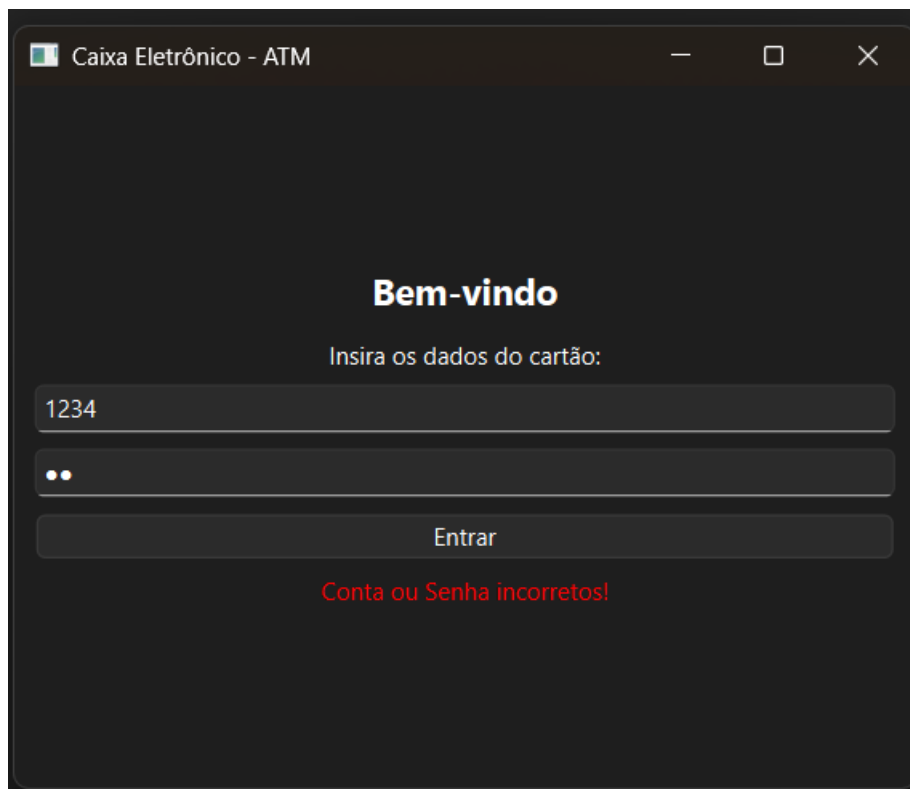
/resources: Agrupa os arquivos estáticos, convertidos em binários pelo sistema .qrc do Qt. Isso inclui a folha de estilo fluent_design.qss, responsável por aplicar as regras de design do Windows 11 (cantos arredondados, paleta de cores correta e fontes Segoe UI).

4 AVALIAÇÃO

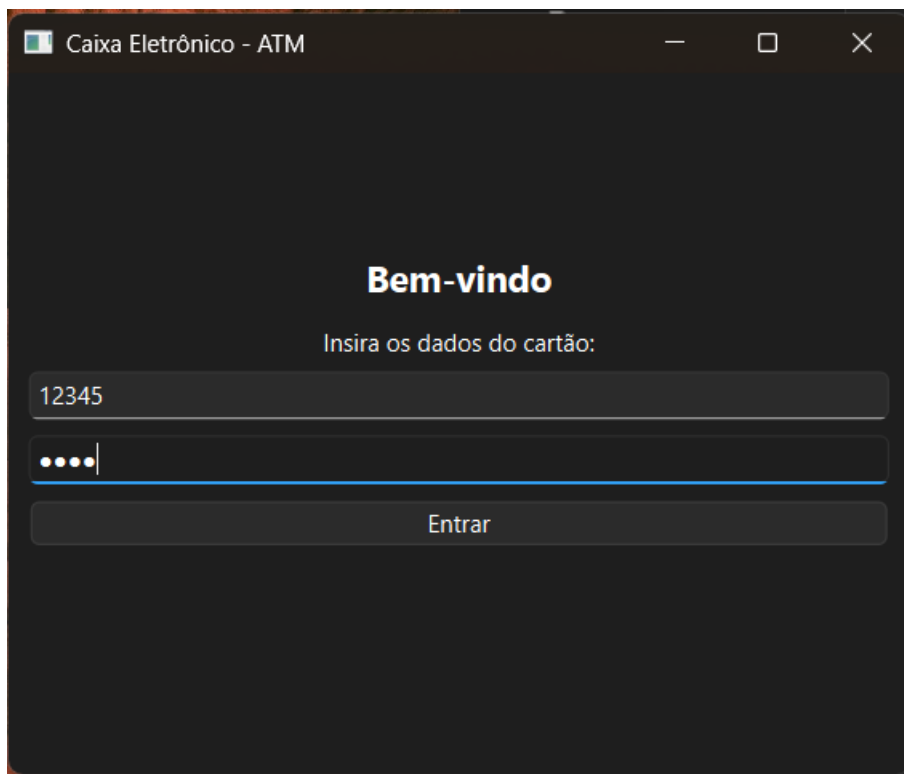
Nesta seção são apresentados os resultados da simulação do sistema e uma discussão sobre a eficácia da solução desenvolvida.

4.1 Condução

A avaliação do sistema foi conduzida através de testes de usabilidade e testes funcionais simulando o comportamento de um cliente do banco. O fluxo avaliado incluiu a inserção do cartão, tentativas de erro de PIN, execução sequencial de múltiplas transações (saque seguido de pagamento de fatura) e a verificação do extrato para garantir o cálculo correto do saldo.



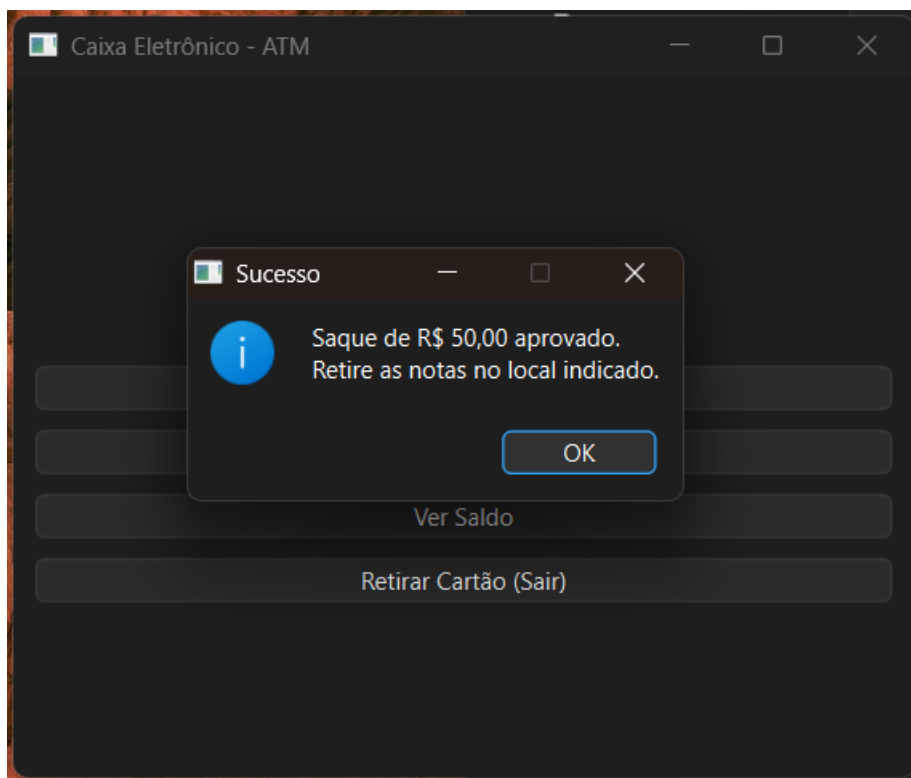
Captura 1: Usuário coloca senha Incorreta e é barrado o acesso à conta bancária



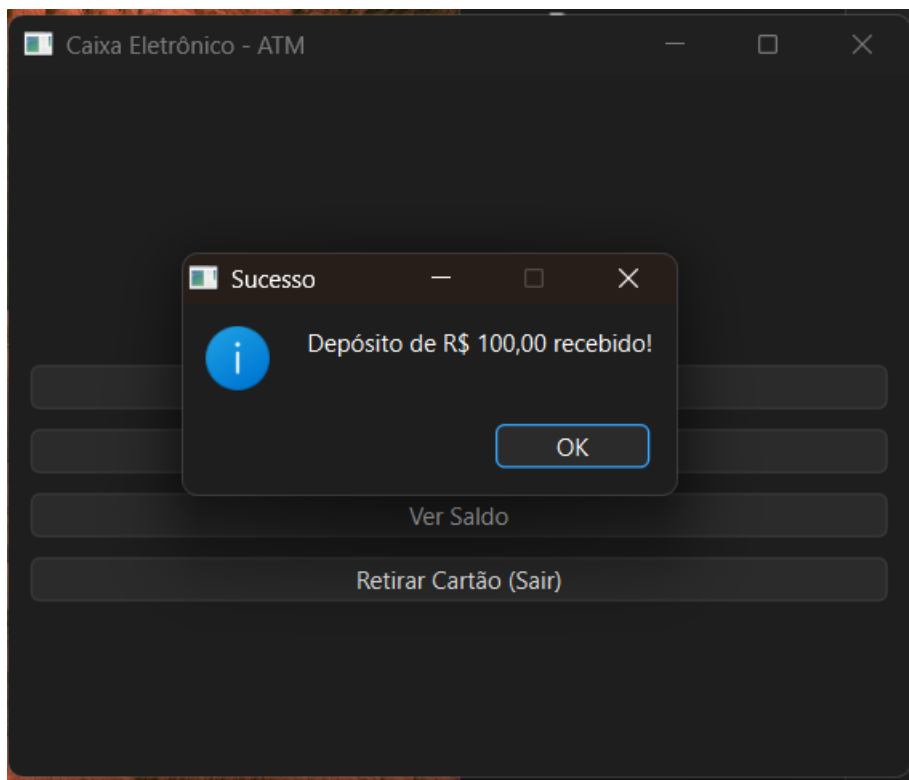
Captura 2: Inserção de dados do Cartão do Cliente



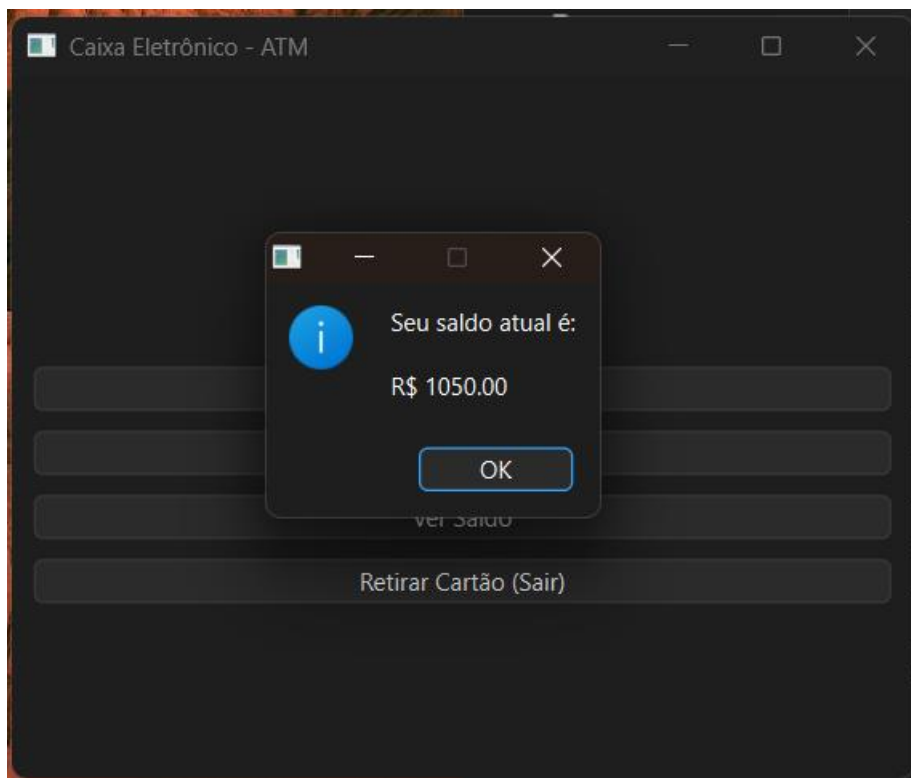
Captura 3: Dados do Cartão Inseridos com Sucesso e Acesso ao menu do Usuário



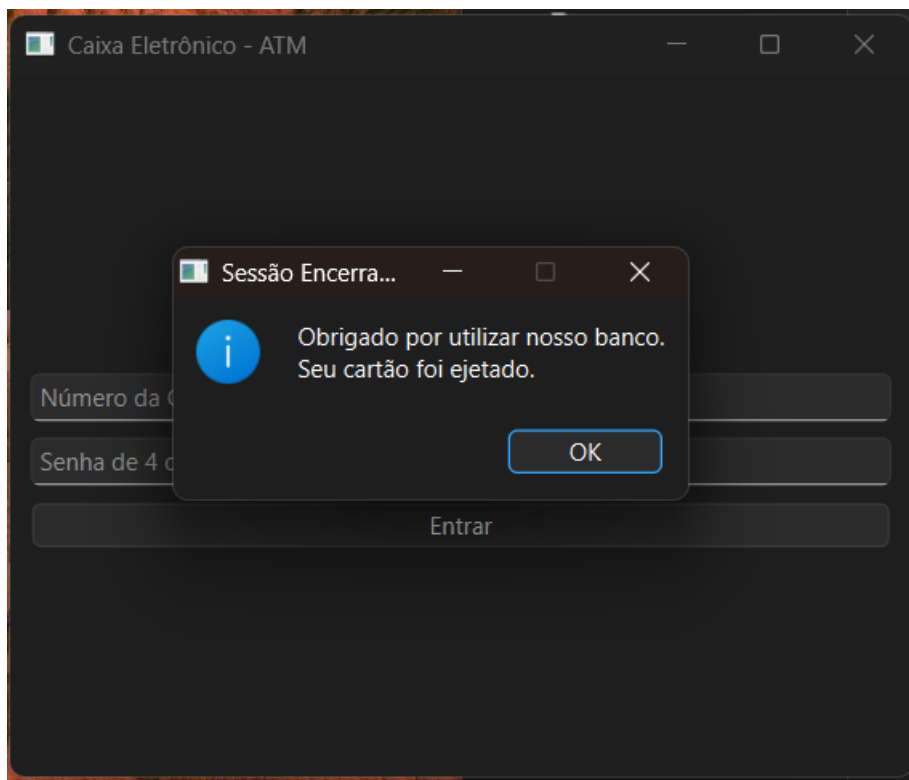
Captura 4: Tela de Saque, o Correntista Retira dinheiro do Slot do Caixa Eletrônico



Captura 5: Tela de Depósito de Dinheiro no Caixa Eletrônico



Captura 6: Tela de Saldo atual após Depósito ou Saque de dinheiro



Captura 7: Tela de Encerramento e Retirada de Cartão do Cliente.

4.2 Resultados

A execução do código demonstrou que a máquina de estados implementada evita com sucesso travamentos ou operações não autorizadas.

A interface reagiu corretamente aos comandos:

Na ocorrência de um PIN inválido, o sistema reteve o usuário na tela de login e emitiu avisos de erro.

Em operações de pagamento de conta (BillPayment), códigos de barras que não obedeciam ao padrão regex de 47/48 dígitos foram sumariamente rejeitados com lançamento de exceções, impedindo débitos incorretos.

O histórico transacional (QTableWidget) refletiu as alterações de saldo de forma cronológica e atômica.

4.3 Discussão

Os resultados confirmam a adequação da linguagem C++11 acoplada ao framework Qt para este escopo. A utilização de ponteiros inteligentes (`std::unique_ptr`) no gerenciamento das transações mitigou o risco de vazamento de memória (memory leaks) que frequentemente afligem aplicações em C++ que rodam por longos períodos (como um ATM real). Dada as conclusões iniciais, a persistência de dados em arquivos binários atendeu aos requisitos imediatos, porém, para escalabilidade futura, o uso do módulo QSql com bancos SQLite ou PostgreSQL representa o próximo passo lógico.

5 CONCLUSÃO

O desenvolvimento do simulador de terminal bancário atingiu todos os objetivos propostos. A síntese dos resultados aponta que a integração rigorosa de conceitos de orientação a objetos com uma interface gráfica orientada a eventos gerou um software estável, responsivo e seguro. A aplicação prática da folha de estilos (QSS) demonstrou que é plenamente viável construir aplicações com visual moderno (Windows 11) diretamente em C++, rompendo com o paradigma de que interfaces nesta linguagem possuem aspecto datado. Como sugestões de melhorias, indica-se a implementação de comunicação via rede (APIs) para validação centralizada de dados e a adição de algoritmos criptográficos para o armazenamento seguro dos hashes de PIN no arquivo de base de dados.

6. Códigos Comentados:

1. Arquivo de Configuração: ATM_Project2.pro

```
//ARQUIVO DE CONFIGURAÇÃO DO QMAKE (O "Motor" do Projeto)
```

```
Adiciona as bibliotecas principais e de interface gráfica do Qt
```

```
QT += core gui
```

```
Se a versão do Qt for maior que 4, adiciona o módulo de widgets
```

```
greaterThan(QT_MAJOR_VERSION, 4): QT += widgets
```

Requer compiladores compatíveis com C++17

CONFIG += c++17

O nome do seu arquivo executável final (.exe)

TARGET = ATM_Project2

Diz que é um aplicativo (app) e não uma biblioteca

TEMPLATE = app

Aqui listamos todos os nossos arquivos de código principal C++ (.cpp)

SOURCES +=

main.cpp

mainwindow.cpp

Aqui listamos todos os nossos arquivos de cabeçalho (.h)

HEADERS +=

mainwindow.h

Aqui listamos os arquivos de design visual (.ui) gerados pelo Qt Designer

FORMS +=

mainwindow.ui

2. Ponto de Partida: main.cpp

```
//  
=====
```

```
// ARQUIVO PRINCIPAL (main.cpp)
```

```
//  
=====
```

```
#include "mainwindow.h"
```

```
#include <QApplication>
```

```
int main(int argc, char *argv[])
```

```
{
```

```
// Inicia o motor principal do aplicativo Qt
QApplication a(argc, argv);

// Cria a janela principal do nosso Caixa Eletrônico na memória
MainWindow w;

// Faz a janela aparecer fisicamente na tela
w.show();

// Mantém o programa rodando em "loop" até o usuário clicar no 'X' para fechar
return a.exec();
}
```

3. Cabeçalho da Interface: mainwindow.h

// ARQUIVO DE CABEÇALHO (mainwindow.h)

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H
```

```
#include <QMainWindow>
#include <QLabel>
#include <QPushButton>
#include <QLineEdit>
#include <QVBoxLayout>
```

```
QT_BEGIN_NAMESPACE
namespace Ui { class MainWindow; }
```


QT_END_NAMESPACE

```
class MainWindow : public QMainWindow
```

```
{
```

```
    Q_OBJECT
```

```
public:
```

```
    // Construtor: Chamado quando a janela é criada
```

```
    MainWindow(QWidget *parent = nullptr);
```

```
    // Destrutor: Chamado quando a janela é destruída/fechada
```

```
    ~MainWindow();
```

```
private slots:
```

```
    // Slots são as funções que respondem a cliques de botões
```

```
    void depositar();
```

```
    void sacar();
```

```
    void atualizarTela();
```

```
private:
```

```
    Ui::MainWindow *ui;
```

```
    // Variável responsável por guardar o dinheiro do Caixa Eletrônico
```

```
    double saldo;
```

```
    // Elementos visuais que criaremos no código
```

```
    QLabel *labelSaldo;
```

```
    QLineEdit *inputValor;
```

```
    QPushButton *btnDepositar;
```

```
        QPushButton *btnSacar;
};
#endif // MAINWINDOW_H
```

4. A Lógica e a Interface: mainwindow.cpp

```
// ARQUIVO DE IMPLEMENTAÇÃO (mainwindow.cpp)

#include "mainwindow.h"
#include "ui_mainwindow.h"
#include <QMessageBox> // Biblioteca para exibir janelas de aviso (pop-ups)

MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
    , saldo(1000.0) // Iniciamos o caixa com um saldo de R$ 1000,00 para testes
{
    ui->setupUi(this);

    // 1. Configurando as propriedades da janela principal
    this->setWindowTitle("Caixa Eletrônico (ATM) - Meu Banco");
    this->resize(400, 300); // Largura x Altura

    // 2. Criando um Widget central para organizar tudo na tela
    QWidget *central = new QWidget(this);
    this->setCentralWidget(central);

    // 3. Criando um Layout Vertical (organiza os itens de cima para baixo
    automaticamente)
```

```
QVBoxLayout *layout = new QVBoxLayout(central);
```

```
// 4. Configurando o texto de exibição do Saldo
```

```
labelSaldo = new QLabel(this);
```

```
labelSaldo->setAlignment(Qt::AlignCenter);
```

```
labelSaldo->setStyleSheet("font-size: 22px; font-weight: bold; margin-top: 20px;  
color: #003366;");
```

```
layout->addWidget(labelSaldo);
```

```
// 5. Configurando a caixinha de texto onde o usuário digita os valores
```

```
inputValor = new QLineEdit(this);
```

```
inputValor->setPlaceholderText("Digite o valor desejado (Ex: 150.50)");
```

```
inputValor->setAlignment(Qt::AlignCenter);
```

```
inputValor->setMinimumHeight(35); // Deixa a caixinha um pouco mais alta
```

```
layout->addWidget(inputValor);
```

```
// 6. Configurando os Botões de Ação
```

```
btnDepositar = new QPushButton("Realizar Depósito", this);
```

```
btnSacar = new QPushButton("Realizar Saque", this);
```

```
// Deixando os botões maiores e mais fáceis de clicar
```

```
btnDepositar->setMinimumHeight(40);
```

```
btnSacar->setMinimumHeight(40);
```

```
// Adicionando os botões na tela (no layout)
```

```
layout->addWidget(btnDepositar);
```

```
layout->addWidget(btnSacar);
```

```
// Conectando o clique do botão com a função matemática correspondente
```

```

connect(btnDepositar, &QPushButton::clicked, this, &MainWindow::depositar);
connect(btnSacar, &QPushButton::clicked, this, &MainWindow::sacar);

// Atualiza o texto na tela pela primeira vez ao abrir o programa
atualizarTela();
}

MainWindow::~MainWindow()
{
    // Limpa a memória quando fechamos o aplicativo
    delete ui;
}

// -----
// FUNÇÕES DE LÓGICA DO CAIXA ELETRÔNICO
// -----

void MainWindow::atualizarTela()
{
    // Pega o valor da variável 'saldo', formata com 2 casas decimais ('f', 2)
    // e coloca como texto no Label do saldo
    labelSaldo->setText(QString("Saldo Disponível: R$ %1").arg(saldo, 0, 'f', 2));
}

void MainWindow::depositar()
{
    bool verificador;

    // Pega o texto digitado na caixinha e tenta transformar em um número fracionado
    (double)

```

```

double valor = inputValor->text().toDouble(&verificador);

// Se a conversão deu certo (é um número) E o valor digitado for maior que zero
if (verificador && valor > 0) {
    saldo = saldo + valor; // Adiciona o dinheiro ao saldo atual
    atualizarTela();      // Atualiza a tela para mostrar o novo saldo
    inputValor->clear();  // Limpa a caixinha de digitação para o próximo uso

    // Exibe um Pop-Up de sucesso
    QMessageBox::information(this, "Sucesso", "Depósito realizado com
sucesso!");
} else {
    // Exibe um Pop-Up de erro se digitarem letras ou números negativos
    QMessageBox::warning(this, "Erro de Validação", "Por favor, digite um valor
válido.");
}
}

void MainWindow::sacar()
{
    bool verificador;

    // Pega o texto digitado na caixinha e tenta transformar em número
    double valor = inputValor->text().toDouble(&verificador);

    // Verifica se digitou um número válido e maior que zero
    if (verificador && valor > 0) {

        // Verifica se há dinheiro suficiente no Caixa para cobrir o saque
        if (valor <= saldo) {

```

```

        saldo = saldo - valor; // Desconta o dinheiro do saldo atual
        atualizarTela();
        inputValor->clear();

        QMessageBox::information(this, "Sucesso", "Saque realizado! Retire as
cédulas.");
    } else {
        // Se o usuário tentar sacar mais do que tem disponível

        QMessageBox::critical(this, "Saldo Insuficiente", "Você não possui saldo para
este saque.");
    }

} else {
    // Exibe um alerta se o valor for inválido (letras, zero ou negativo)

    QMessageBox::warning(this, "Erro de Validação", "Por favor, digite um valor
válido para sacar.");
}
}

```

REFERÊNCIAS

DEITEL, Paul; DEITEL, Harvey. C++ Como Programar. 10. ed. São Paulo: Pearson, 2017.

GAMMA, Erich et al. Padrões de Projetos: Soluções Reutilizáveis de Software Orientado a Objetos. Porto Alegre: Bookman, 2000.

STROUSTRUP, Bjarne. A Linguagem de Programação C++. 4. ed. Porto Alegre: Bookman, 2014.

THE QT COMPANY. Qt Documentation. Disponível em: <https://doc.qt.io/>. Acesso em: 05 jun. 2026.

GLOSSÁRIO

Regex: Sequência de caracteres que forma um padrão de busca, utilizado neste projeto para validar o comprimento e o formato numérico dos códigos de barras de boletos.

Ponteiros inteligentes: Recursos do C++11 (como `std::unique_ptr`) que automatizam o gerenciamento de memória, garantindo que objetos alocados sejam destruídos corretamente quando não estão mais em uso.

APÊNDICE A: CÓDIGO FONTE BASE DE TRANSAÇÃO

Algoritmo 1 - Interface Polimórfica de Transação em C++11

```
#ifndef TRANSACTION_H
#define TRANSACTION_H

class Transaction {
protected:
    int accountNumber;
    BankDatabase* db;
public:
    Transaction(int acc, BankDatabase* database)
        : accountNumber(acc), db(database) {}
    virtual ~Transaction() = default;

    // Método virtual puro que garante o polimorfismo
    virtual void execute() = 0;
};

#endif
```